Solving the Beck and Wieland Model with Optimal Experimentation in *DualPC* [1]

Hans M. Amman, David A. Kendrick, Marco P. Tucci

Utrecht School of Economics, Utrecht University, Heidelberlaan 8, 3584 CS Utrecht, the Netherlands.

Department of Economics, University of Texas, Austin, Texas 78712, USA.

Dipartimento di Economia Politica, Università di Siena, Piazza S. Francesco, 7, 53100 Siena, Italy

Abstract

Currently, there is a renewed interest in the use of optimal experimentation (adaptive control) in economics. Example are found in Amman and Kendrick [3,4], Casimano [6], Casimano and Gapen [8,7,9], Tucci [17] and Wieland [20,21]. In this paper we present the Beck and Wieland model [5] and the methodology to solve this model with *time varying parameters* using the various control methods described in Kendrick [11,12]. Furthermore, we also provide numerical results using the *DualPC* software [2] and show first evidence that optimal experimentation or *Dual Control* may produce better results then *Expected Optimal Feedback*.

Key words: Dual Control, optimal experimentation, stochastic optimization, time-varying parameters, numerical experiments.

1 Introduction

This paper is a part of the Methods Comparison Project to compare various methods for solving economic models through optimal experimentation. [2] Our

Email addresses: amman@uu.nl (Hans M. Amman), kendrick@eco.utexas.edu (David A. Kendrick), tucci@unisi.it (Marco P. Tucci).

[1] Corresponding author Hans M. Amman, document name *dualpc.tex*.

[2] A complementary paper [14] provides a short description of the project, of the methods, and of the Beck and Wieland [5] model to which all of the methods are being applied. It also provides the numerical results using the *DualI* software [2] while this paper provides the numerical results using the *DualPC* software [1].

purpose in this paper is three fold. First, we want to provide two versions of the Beck and Wieland [5] model (BWM), as a basis for comparing numerical results across methods and codes. Up to now, there have been very few adaptive control codes applied to economic models. As such, there has not been much opportunity to compare numerical results across codes and thus little opportunity to gain comparative assurance that those results are correct.

The second purpose of this paper is to provide a quantitative analysis of the properties of the BWM when it is solved with adaptive control methods. For us this means a comparison of simple to sophisticated solution methods across Monte Carlo runs to ascertain the comparative performance of the different methods. For example, we compare Optimal Feedback (*OF*), Expected Optimal Feedback (*EOF*) and Dual Control (*DC*) methods as applied to the two versions of the BWM. [3] The two versions of the model differ according to whether the true parameters are *constant* or *time-varying*.

Our third purpose is to provide results that can be used to begin to analyze the comparative advantage of the various solution methods for adaptive control models. Thus we hope these results provide a foundation for comparing the accuracy and speed of the various methods and their ability to handle non-convexities in the cost-to-go function. Also, we hope to shed more light on whether or not non-convexities in the cost-to-go functions will prove to be common or unusual with the range of parameter values that are used in economic models.

In working toward the achievements of these three purposes we have used two software systems. The first is *DualI* (pronounced Dual Eye) [2] and the second is *DualPC* [4] [1]. The *DualI* software is written in the C language and will, when completed, provide a graphical interface in Windows for the *DC* method, thus the name *DualI*. *DualI* is designed to lower the entry cost for students and researchers who have an interest in solving stochastic control models. *DualI* supports *OF* and *EOF* methods but does not yet contain *DC* methods.

In contrast, the *DualPC* software is written in Fortran with a much older interface; however, it contains *DC* as well as *OF* and *EOF* methods. It is implemented for both constant and time-varying parameters. However, the types of time-varying parameter specifications supported in *DualPC* are limited for the *EOF* case. For more varied specifications see Tucci [16–18] and Tucci et

[3] For a description of the various methods, see also Kendrick and Amman [13].
[4] A copy of either *DualPC* or *DualI* can be obtained, for non commercial use, by sending a request to the corresponding author.

al. [19]. [5]

We begin with a brief presentation of the BWM followed by the results for the constant parameter version of the model and then progress to the results for the version with *time-varying* parameters.

2 Outline of the Beck and Wieland Model

Following Beck and Wieland [5] the decision maker is faced with a linear stochastic optimization problem of the form [6]

$$\underset{[u_t]_{t=0}^{T-1}}{Min}\ E\Bigg[\delta^T(x_T-\hat{x}_T)^2+\sum_{t=0}^{T-1}\delta^t\{(x_t-\hat{x}_t)^2+\omega(u_t-\hat{u}_t)^2\}\mid(x_0,b_0,v_0^b)\Bigg] \tag{1}$$

subject to the equations

$$x_{t+1}=\alpha+\beta_t u_t+\gamma x_t+\epsilon_t \tag{2}$$
$$\beta_{t+1}=\beta_t+\eta_t \tag{3}$$

where $\epsilon_t \sim N(0,\sigma_\epsilon)$ and $\eta_t \sim N(0,\sigma_\eta)$. The model contains one uncertain parameter β, with an initial estimate of its value at $t=0$, $E_0(\beta_0)=b_0$, and an initial estimate of its variance at time $t=0$, $\sigma_{\beta_0}=v_0^b$. The parameters α and γ are constant. Beck and Wieland assume in their paper that $T\to\infty$. In contrast we will assume that the planning horizon is finite, hence $T<\infty$.

[5] The *EOF* part of *DualPC* is based on the numerical example discussed in Kendrick [11, Chapter 7]. As such the software *DualPC* does not cover the full fledged time varying case for *EOF* with $\beta_{t+1}=D_t\beta_t+\eta_t$ where $D_t\neq I$ and $\sigma_\eta>0$. In Tucci et al. [19], the derivation of the *EOF* solution with time varying parameters is presented and it is shown that, for the model and data set used in Beck and Wieland [5] and in the present work, it collapses to the *EOF* method presented in Kendrick [11]. This is due to the fact that in equation (3) the time varying process follows a random walk, with $D=I$, and there is no penalty attached to the controls. Therefore the *EOF* for the BWM and data set, is identical to the obtained using the method in Kendrick [11, chapter 6].

[6] In an earlier strand of literature, going back to the early Seventies, a similar model and approach is discussed. See, MacRea [15].

Furthermore, we have adopted the "timing" convention from Kendrick [11,12] where the control, u_t, has a lagged response on the state, x_t.

For the simulations in the next paragraphs we will use the following numerical values for experiments: $\alpha = 0$, $b_0 = -0.50$, $v_0^b = 1.25$, $\gamma = 1$, $\sigma_\epsilon = 1$, $\sigma_\eta = 0$ for the constant parameter case and $\sigma_\eta = 0.04$ for the time varying parameter case (see the next two sections). Furthermore, $\omega = 0$, $T = 10$, $\forall t$ $\hat{x}_t = 0$, $\forall t$ $\hat{u}_t = 0$, $\delta = 1$, $x_0 = 1$.

With this set of parameters, the above model can be solved in *DualPC* [14], allowing us to simulate the various situations in the next sections.

3 Constant Parameters

We make a distinction, at each time step, between the *true* parameters, β_t, and the *estimates* of those parameters, b_t. In this version of the model the true values of the parameters are constant but the estimates change over time. In contrast, in the time-varying parameter version of the model, Section 4, both the true parameters and the estimates change over time.

The echo print of the input file for this constant parameter version of the model is in subsection A.1 of the Appendix. The parameters are the same as those used with the versions of the model solved with *DualI* except for the discount factor, δ, which is set at 0.95 in the *DualI* versions and at 1.00 in the *DualPC* versions. The reason for this is that the *DualPC* [7] software does not yet support discounting.

We used the *DualPC* software to run 10,000 Monte Carlo in which we compared the criterion values obtained with three different methods *OF*, *EOF* and *DC*. As indicated above, the first two methods are described in the complementary paper [14]. The *DC* (adaptive control) methods used here is the one described in Chapters 9 and 10 of Kendrick [11,12]. In addition, the *DualPC* software includes a grid search method that is designed to deal with possible non-convexities in the cost-to-go function. This is a two level grid search that begins in the neighborhood of the *OF* solution. The best grid point obtained in the first search then provides the starting point for the second level search which is done in finer detail over a lesser range than the first grid search.

When we applied the *OF*, *EOF* and *DC* methods to the BWM we found that in a substantial number of the runs the criterion value for one or more of the methods was unusually large. Or, to say this in another way, the distribu-

[7] See also footnote (5).

tion of criterion values had a long right tail.

The outlier problem may be caused by the initial parameter estimates for the uncertain parameter, which are themselves outliers in either the right or left tails of that distribution. Assuming that the uncertain parameter has mean -.5 and variance 1.25 implies that the initial value used in the Monte Carlo runs is in the interval (-1.62, .62) approximately 68% of the cases, and in the intervals (-2.74, 1.74) and (-3.86, 2.86) approximately 95% and 99%, respectively, of the cases. Alternatively put, the initial estimate of the unknown parameter is outside the 'benevolent' interval (-1.62,.62) approximately 32% of times and this is obviously reflected in the associated value of the criterion function. To see how the various methods perform in the different situations we decided to run the comparison three times. In the first test we kept the runs in which the criterion value for all of the three methods was less than or equal to 100. In the second test we set this boundary at 200 and in the third run we set the boundary at 500. Thus in the three test we include a larger and larger number of the runs that we are uneasy about. Therefore we are inclined to give more credence to the test with the lower cut off values.

Our calculations were performed on a 64 bit AMD Dual Opteron Linux machine. Each simulation run of 10,000 Monte Carlo runs took about an hour of processing time on one processor. In comparing the results we looked first at the percentage of runs in which each method proved to have the lowest criterion value. These results are shown in Table 1.

Table 1

Percentage of Runs in Which Each Method had the Lowest Criterion

J	OF	EOF	DC
$J < 100$	44.2	18.1	37.7
$J < 200$	43.0	18.4	38.6
$J < 500$	41.7	19.0	39.2

The first line of Table 1 shows that for the $J < 100$ case the percentage of Monte Carlo runs for which each method obtained the lowest criterion value was *OF* 44 percent, *EOF* 18 percent and *DC* 38 percent. These results proved to be relatively constant across the three rows of Table 1 which indicated that the outlier problem does not seem to affect the relative performance of the three methods.

The second way we compared the results was by examining the average criterion value for each method with their standard errors. [8] These results are

[8] The standerd errors are printed between parentheses and defined as $\frac{s}{\sqrt{n}}$ where s

shown in Table 2.

Table 2

Average Criterion Value for Each Method

J	OF	EOF	DC
$J < 100$	14.247	17.527	10.979
	(0.158)	(0.163)	(0.103)
$J < 200$	18.021	18.627	11.732
	(0.267)	(0.188)	(0.133)
$J < 500$	25.970	18.803	12.142
	(0.564)	(0.188)	(0.155)

The first row in Table 2 shows that the *OF* and *EOF* method do not do as well as the *DC* methods in this $J < 100$ case. This is the result to which we currently assign the most credence because it describes a situation where the estimated parameter is not "too far" from the true unknown value. Looking down the columns in Table 2 we see, not unexpectedly, that the average criterion values increase as more of the outliers are included. However, it is worth noting that the criterion values in the *EOF* column do not increase as rapidly as do those in the OF column. This is consistent with our results in [3], that when the *OF* solutions are bad they may be really bad, i.e. when you have an outlier estimate for the parameter value and treat it as though you trust that it is correct one can get a seriously bad solution. In these cases *EOF* is better, because it is cautious, and DC is better yet, because it devotes a part of the control energy to experiments to help learn the parameter value. Next we turn to the version of the BWM with time-varying parameters.

4 Time-Varying Parameters Version

In this version of the BWM the true value of the parameter is time varying and follows a first order Markov process. The echo print of the input file for this version of the model is in subsection A.2 of the Appendix. The major change from the first version of the model is that the variance of the additive noise term in the parameter evolution equation, σ_η is not zero, as in the previous version, but is 0.04. Also, recall that for both versions of the model solved with the *DualPC* software, the discount factor, δ, is not 0.95 but rather 1.00.

is the standard deviation in the sample and n the number of monte carlo runs. With the help of the standard errors it is possible to compute the confidence intervals for the various methods. For instance, the 95% procent confindence for the mean of the *DC* solution is $10.979 \pm 2 \times 0.103$, see for instance Glasserman [10, page 541].

Just as with the constant parameter version of the model, in comparing the results we looked first at the percentage of the 10,000 Monte Carlo runs in which each method proved to have the lowest criterion value. These results are shown in Table 3.

Table 3

Percentage of Runs in Which Each Method had the Lowest Criterion

J	OF	EOF	DC
$J < 100$	30.9	28.0	41.1
$J < 200$	29.4	28.9	41.7
$J < 500$	27.9	30.6	41.5

The first line of Table 3 shows that for the $J < 100$ case the *OF* method and the *EOF* method each had the lowest criterion valve in roughly 30 percent of the Monte Carlo runs and that the *DC* method had the lowest criterion value in roughly 40 percent of the runs. So the *DC* method proves to be the best of the three when the comparison is done in this way.

Then a comparison of the second and third rows in Table 3 to each other and to the first row shows that the percentage of the Monte Carlo runs in which each method had the lowest criterion value was not affected much by the number of the outliers that were included. Again the outliers do not seem to affect the relative performance of *OF*, *EOF* and *DC*.

The second way we compared the results was by examining the average criterion value. These results with their standard errors [9] are shown in Table 4.

Table 4

Average Criterion Value for Each Method

J	OF	EOF	DC
$J < 100$	18.612	17.289	12.529
	(0.197)	(0.167)	(0.126)
$J < 200$	26.511	19.362	15.433
	(0.357)	(0.207)	(0.212)
$J < 500$	43.733	20.254	20.493
	(0.788)	(0.219)	(0.423)

[9] The standerd errors are between parentheses. See also footnote 8.

The first row in Table 4 shows that the *OF* method does not do as well as the *EOF* method which in turn does not do as well as the *DC* methods in the $J < 100$ case. This is the result to which we currently assign the most credence.

Looking down the columns in Table 4. shows, not unexpectedly, that the average criterion values increase as more of the outliers are included. However, it is worth noting that the criterion values in the *EOF* column do not increase as rapidly as do those in either of the other columns.

5 Summary

In this paper we have presented evidence that optimal experimentation, *Dual Control*, may produce better results then Expected Optimal Feedback. For this purpose we have used the Beck and Wieland model as a vehicle. Especially, in situations where the (initial) level of uncertainty about the model's parameters is large, experimentation yields better results.

Appendix

A The Beck and Wieland Model in the DualPC Software

This appendix contains the specifications of the two versions of the Beck and Wieland model that were used in this paper. The input files below, enable the user to solve the two models using the *DaulPC* software.

A.1 Constant Parameter Version

This model is input to the *DualPC* software with one state, n, one control, m, one exogenous variable, ns, one uncertain parameter, np and one observation variable, ny. Also, the model is solved for 10 time periods, nt, and for 10,000 Monte Carlo runs, nmc. Most of the notation is self explanatory with the following exceptions:

- flam is the parameter ω in the criterion function
- xdes, $\hat{x}_t$, and udes, $\hat{u}_t$, are the desired paths for the states and controls, respectively
- th0, β_0, is the mean of the uncertain parameter and sitt0 is the variance, $VAR_0(\beta_0)$, of that parameter.

- sixx0, $VAR_0(x_0)$ is the variance of the initial state at $t = 0$ and sitx0, $COV_0(x_0, \beta_0)$, the covariance of this initial state with the intitial value of the unknown parameter at $t = 0$.
- gam is the variance of the additive noise term, σ_η, in the parameter evolution equation
- ithn is the information used to map the elements in the unknown parameter vector to their location in the system equation parameter matrices and vector. In this case the only uncertain parameter is in the B matrix (denoted by 1) and is the first row and first column in that matrix (1,1).

```
/* Wieland Beck model, JEDC, 2002 */
      n     m    ns    np    ny    nt  nmc
      1     1     1     1     1    10 10000
   l1  l2  l3  l4  l5  l6
    1   1   1   1   0   0
   l7  l8  l9  l10 l11 l12
    0   1   0   0   0   0

 Active options during this simulation are:

 l1=1: active learning (dc) simulation.
 l2=1: passive learning (eof) simulation.
 l3=1: sequential updating (of) simulation.
 l4=1: linear-quadratic (qlp) simulation.
 l8=1: print intermediate results on file mcres.
 l9=0: use grid search for m<=2 in dual.

 a
      1.0000,
 b
     -0.5000,
 c
      0.0000,
 d
      1.0000,
 w
      1.0000,
 wn
      1.0000,
 flam
      0.0001,
 exomat
      1.0000,    1.0000,    1.0000,    1.0000,    1.0000,
 exomat
```

```
     1.0000,    1.0000,    1.0000,    1.0000,    1.0000,
xdes0
     0.0000,
xdes
     0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
xdes
     0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
udes
     0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
udes
     0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
h
     1.0000,
x0
     1.0000,
th0
    -0.5000,
sixx0
     0.0001,
sitx0
     0.0001,
sitt0
   0.1250E+01,
q
     1.0000,
r
     0.0001,
gam
     0.0000,

Warning: subroutine cholks returns that the
matrix is not positive definite. it will be
set to zero when used for random generator

ithn
1  1  1
atrue
     1.0000,
btrue
    -0.5000,
ctrue
     0.0000,

One dimensional grid search will be used in dual
```

A.2 Time-Varying Parameter Version

The major change in this version, from the constant parameter version of the model in the first part of this Appendix, is that the variance of the additive noise in the parameter evolution equation, gam (σ_η), is not set to zero but rather to 0.04.

```
/* Wieland Beck model, JEDC, 2002 */

     n     m    ns    np    ny    nt  nmc
     1     1     1     1     1    10 10000
  l1  l2  l3  l4  l5  l6
   1   1   1   1   0   0
  l7  l8  l9  l10 l11 l12
   0   1   0   0   0   0

 Active options during this simulation are:

 l1=1: active learning (dual) simulation.
 l2=1: passive learning (olf) simulation.
 l3=1: sequential updating (ce) simulation.
 l4=1: linear-quadratic (qlp) simulation.
 l8=1: print intermediate results on file mcres.
 l9=0: use grid search for m<=2 in dual.

 a
     1.0000,
 b
    -0.5000,
 c
     0.0000,
 d
     1.0000,
 w
     1.0000,
 wn
     1.0000,
 flam
     0.0001,
 exomat
     1.0000,    1.0000,    1.0000,    1.0000,    1.0000,
 exomat
     1.0000,    1.0000,    1.0000,    1.0000,    1.0000,
 xdes0
```

```
     0.0000,
xdes
     0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
xdes
     0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
udes
     0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
udes
     0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
h
     1.0000,
x0
     1.0000,
th0
    -0.5000,
sixx0
     0.0001,
sitx0
     0.0001,
sitt0
   0.1250E+01,
q
     1.0000,
r
     0.0001,
gam
     0.0400,
ithn
1  1  1
atrue
     1.0000,
btrue
    -0.5000,
ctrue
     0.0000,

One dimensional grid search will be used in dual
```